Low-Level Design (LLD)

Objective

The objective of the Low-Level Design (LLD) is to describe how the system is implemented internally. It explains the modules, their responsibilities, and how they work together to process user queries and generate responses.

1. Module-Level Design

1.1 Document Processing Module

- Responsible for loading the knowledge base
- Reads data from .txt or .pdf files
- Converts content into document objects

1.2 Chunking Module

- Splits large documents into smaller chunks
- Ensures better retrieval accuracy
- Uses fixed chunk size (e.g., 500 characters)

1.3 Embedding Module

- Converts text chunks into vector embeddings
- Uses embedding model (all-MiniLM-L6-v2)
- Enables semantic search

1.4 Vector Storage Module

- Stores embeddings in ChromaDB
- Supports fast similarity search
- Maintains mapping between text and vectors

1.5 Retrieval Module

- Retrieves top-k relevant chunks based on query
- Uses similarity search
- Filters irrelevant results

1.6 Query Processing Module

- Accepts user input
- Converts query into searchable format
- Passes query to retrieval system

1.7 Graph Execution Module (LangGraph)

- Controls the workflow logic
- Defines nodes and transitions
- Handles routing between components

1.8 HITL Module

- Handles escalation to human support
- Triggered when system cannot answer
- Ensures reliability of responses

2. Data Structures

2.1 Document Representation

- Each document is stored as:

```
Document(page_content="Refund will be processed in 5 days")
```

2.2 Chunk Format

- Text is split into smaller chunks:

```
"Refund will be processed in 5 working days"
```

2.3 Embedding Structure

- Each chunk converted into vector:

```
[0.12, -0.45, 0.67, ...]
```

2.4 Query-Response Schema

Input:

```
{
  "query": "refund policy"
}
```

Output:

```
{
  "answer": "Refund will be processed in 5 working days"
}
```

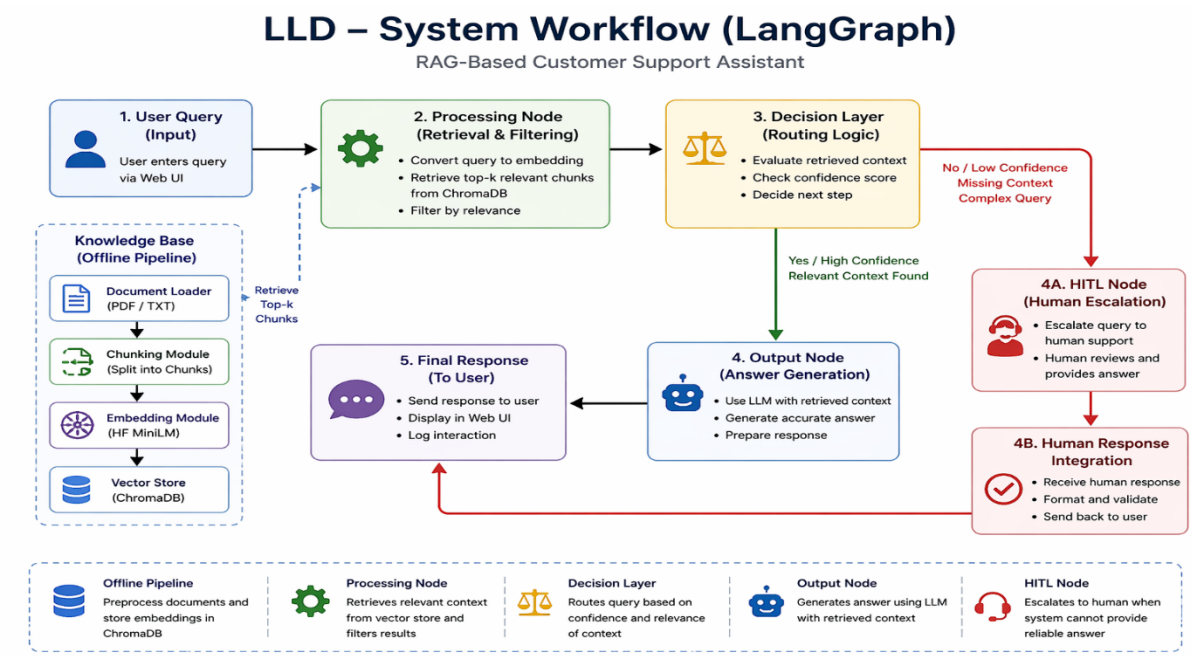
2.5 Graph State Object

```
state = {
    "query": "",
    "context": "",
    "answer": ""
}
```

3. Workflow Design (LangGraph)

The workflow of the system is managed using LangGraph, which controls how the query is processed and how responses are generated or escalated.

Workflow Diagram



Nodes

- **Processing Node** → retrieves relevant data
- **Output Node** → generates final answer

Edges

- Processing → Output
- Conditional edge → HITL if needed

State Flow

1. Query enters system
2. Context is retrieved
3. Answer is generated
4. Output is returned

4. Conditional Routing Logic

Answer Generation Criteria

- Relevant chunks are found

- Context is sufficient

Escalation Criteria

- No relevant data
- Low confidence retrieval
- Missing context
- Complex query

5. HITL Design

When Triggered

- Retrieval returns empty result
- System cannot generate answer

After Escalation

- Query sent to human support
- Human provides response

Integration

- Human answer returned to user
- Can be stored for future improvement

6. API / Interface Design

Input Format

```
{
  "query": "refund policy"
}
```

Output Format

```
{
  "answer": "Refund will be processed in 5 working days"
}
```

Interaction Flow

1. User enters query
2. System processes input
3. Retrieves data
4. Generates answer
5. Displays result

7. Error Handling

Missing Data

- File not found → display error

No Relevant Chunks

- Trigger HITL escalation

LLM Failure

- Return fallback message
- Example:

"Unable to process request. Please try again."